AgentExam: An Upload-Driven Retrieval-Augmented AI Agent for Adaptive Examination Preparation

First A. Author[1], Second B. Author[2], and Third C. Author[1]

[1]Department of Computer Science and Engineering, Institute Name, City, Country

[2]Department of Information Technology, Institute Name, City, Country

Corresponding authors: author1@institute.edu, author2@institute.edu, author3@institute.edu

***Abstract*—Students preparing for university examinations routinely assemble heterogeneous study material—syllabi, lecture notes, previous-year question (PYQ) papers, and laboratory manuals—yet most digital learning platforms presuppose a fixed curriculum and offer little beyond static question banks. We present AgentExam, a full-stack web application that inverts this assumption: the system ships with no predefined subjects and derives the entire learning experience from the material a learner or instructor uploads. An asynchronous ingestion pipeline extracts text from PDF, DOCX, plain-text, and image files, employs a large language model (LLM) to detect the subject and reconstruct its curriculum as units and weighted topics, and organizes every artifact category-wise (syllabus, notes, PYQs, lab manuals) under each subject. A retrieval-augmented generation (RAG) layer chunks all extracted text and applies BM25 lexical retrieval with source- and recency-aware boosting, so that question banks, mock examinations, viva sessions, laboratory reviews, topic-importance analysis, and seven-day study plans are generated from evidence retrieved from the learner's own corpus. A faculty mode extends the platform with classroom management, shared material libraries, assigned tests, and per-student progress analytics. We report measured behavior of the deployed system: sub-second upload acknowledgment, 20–25 s background analysis per document, verbatim extraction of 35 previous-year questions from authentic university papers, and weak-topic detection that surfaced eight actionable deficit areas. AgentExam demonstrates that a corpus-grounded, upload-driven agent can deliver adaptive assessment without any predefined curriculum.**

***Index Terms*—Adaptive assessment, artificial intelligence in education, BM25 retrieval, examination preparation, large language models, previous-year question analysis, question generation, retrieval-augmented generation.**

I. Introduction

Preparation for university examinations is, in practice, a material-centric activity. A syllabus defines the scope of a course; lecture notes and textbooks supply depth; previous-year question (PYQ) papers reveal the priorities of past examiners; and laboratory manuals define the practical component. A student enrolled in five or six courses in a semester manages dozens of such files across formats—PDF syllabi, word-processed notes, scanned question papers, and photographed handouts—usually scattered across folders and messaging applications.

Existing digital learning platforms rarely match this reality. Most presuppose a fixed catalog of subjects and serve static question banks that are disconnected from the particular institution's syllabus, the instructor's emphasis, or the learner's own material. PYQ analysis, where it matters most, is performed by hand: students skim old papers, guess which topics recur, and annotate their notes accordingly. Formative feedback is delayed until graded examinations are returned. On the other side of the classroom, faculty who assign practice tests have limited tooling for sharing material with a cohort and even less for observing per-student progress.

We present *AgentExam*, a full-stack web application that inverts the fixed-curriculum assumption. The system initializes with an empty account: no subjects, no material, no question bank. When a user uploads a document—of virtually any study-oriented format—an asynchronous pipeline extracts its text, an LLM detects the subject and reconstructs its curriculum as units and topics annotated with importance, PYQ frequency, and weightage, and a retrieval-augmented generation (RAG) layer builds a question bank grounded in the uploaded corpus. Every subsequent capability—mock examinations, viva practice, laboratory reviews, PYQ analytics, weak-topic detection, and a seven-day study plan—is derived from and referenced against that corpus. A faculty mode adds classrooms, shared material libraries, assigned tests, and per-student analytics.

The contributions of this work are as follows:

1) An upload-driven architecture in which no subject, unit, or topic exists until a learner or instructor uploads material; the curriculum is reconstructed by an LLM from the artifacts themselves.
2) A resilient, asynchronous ingestion pipeline—multi-format text extraction including vision-model OCR for scanned PDFs, bounded-concurrency background queueing, and retry with exponential backoff—that acknowledges uploads in approximately 0.3 s and completes analysis in 20–25 s.
3) Category-wise material organization: every upload is classified as syllabus, notes, PYQ paper, or lab manual, and the interface groups artifacts by subject and category so that five or six concurrent subjects remain navigable.
4) A RAG layer that chunks the full extracted corpus per subject and applies BM25 lexical retrieval with source- and recency-aware boosts, grounding question banks, PYQ analytics, viva questions, laboratory reviews, and study plans in retrieved evidence.
5) Weak-topic detection with calibrated status thresholds and AI-first seven-day study planning that accounts for topic weightage, PYQ frequency, detected deficits, and the learner's target examination date.

6) A faculty workflow covering classroom creation with join codes, student provisioning, shared uploads, test conduction with per-student result grouping, and cohort analytics, together with a deployment evaluation reporting measured behavior on authentic university material.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture and data model. Section IV details the upload-driven material processing pipeline, and Section V the retrieval-augmented generation layer. Section VI presents the adaptive assessment and analytics capabilities. Sections VII and VIII describe the implementation and its evaluation, Section IX discusses limitations, and Section X concludes.

II. Related Work

Intelligent tutoring systems. Tutoring systems have long sought to close the feedback loop that classroom instruction leaves open. VanLehn [5] characterized tutoring systems by the granularity of their decision loops and found that the effectiveness of a tutor depends less on its surface sophistication than on the quality and timing of its feedback. Formative assessment research reaches a compatible conclusion: Black and Wiliam [8] showed that frequent, specific feedback is among the strongest levers on learning outcomes. AgentExam adopts this principle operationally—every mock examination, viva answer, and laboratory submission is evaluated immediately with per-question feedback—while replacing hand-authored content with corpus-derived content.

Automatic question generation. Kurdi et al. [6] surveyed automatic question generation for education and identified template- and rule-based methods as the historical mainstream, with semantic correctness and pedagogical alignment as persistent open problems. Recent instruction-tuned LLMs [3], [4] have made free-form generation fluent, but fluency does not guarantee fidelity to a course. Our approach addresses fidelity structurally: generation is conditioned on retrieved chunks of the learner's own material, and a validation stage rejects or regenerates malformed banks rather than persisting them.

Large language models in education. Kasneci et al. [9] catalogued both the opportunities and the risks of deploying LLMs in educational settings, with hallucination and over-reliance as principal hazards. Chain-of-thought prompting [11] and structured output constraints mitigate but do not eliminate these risks. AgentExam therefore treats the LLM as a fallible component: every generative path has a deterministic fallback, extraction quality is measured rather than assumed, and faculty can review and re-analyze any artifact.

Retrieval-augmented generation. Lewis et al. [1] demonstrated that interleaving retrieval with generation substantially improves factual grounding for knowledge-intensive tasks. Retrieval itself spans classical lexical scoring—BM25 and its descendants [2]—and dense embedding-based passage retrieval [12]. AgentExam uses BM25 over chunked course material: the corpora are small (thousands of chunks, not millions of passages), determinism and auditability are valuable in an assessment context, and the lexical index requires no additional model infrastructure. Retrieval scores are additionally biased toward authoritative sources (syllabus) and recent uploads, a policy akin to document priors in probabilistic relevance frameworks [2].

Adaptive scheduling. Spaced-repetition schedulers, such as the trainable model deployed at scale by Settles and Meeder [7], demonstrate that timing practice to learner state improves retention. AgentExam's seven-day study planner is a lightweight instantiation of the same idea: task selection is driven by topic weightage, PYQ frequency, and detected weaknesses rather than a fixed rotation.

In combination, prior work supplies the components—grounded generation, immediate feedback, adaptive scheduling—but not the integrated loop in which the learner's own uploaded corpus is the single source of truth for curriculum structure, question content, and analytics. Closing that loop, and reporting the engineering behavior of doing so on authentic material, is the subject of this paper.

III. System Overview and Architecture

A. Architectural Overview

AgentExam is organized into four layers (Fig. 1). The *presentation layer* is a single-page web client that switches among fourteen role-aware views (onboarding dashboard, materials, syllabus analysis, PYQ analysis, theory examinations, viva, practical, performance, readiness, study plan, profile, faculty dashboard, classroom detail, and authentication views) without page reloads. The *application layer* exposes 27 REST endpoints as server-side route handlers, each guarded by bearer-token authentication and role checks. The *intelligence layer* hosts the hosted LLM and vision-language services accessed through a server-side SDK, the RAG engine, and an in-process job queue that bounds pipeline concurrency. The *data layer* comprises an ORM-mapped relational database and an external document store for uploaded files.

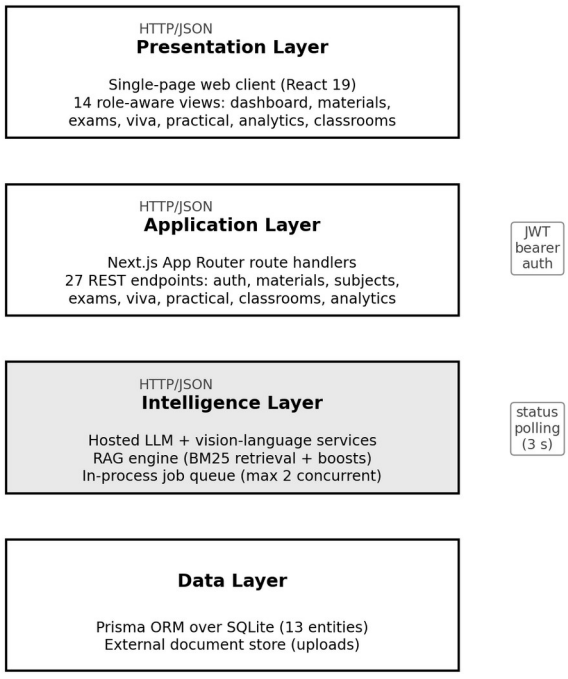


Fig. 1. Layered architecture of AgentExam. The intelligence layer hosts the LLM and vision-language services, the retrieval-augmented generation engine, and the bounded-concurrency job queue.

Two cross-cutting interactions merit note. First, all authenticated requests carry a JSON Web Token; the application layer resolves the user on every request and enforces subject-level visibility before any data is returned. Second, because material analysis is asynchronous, the client polls a status endpoint at three-second intervals while any material is in a processing state, and transitions in the polled state drive user-facing notifications.

B. Data Model

The relational schema centers on the curriculum hierarchy *Subject → Unit → Topic*, where each topic carries an importance rating, a PYQ-frequency estimate, and an examination weightage. Uploaded material is modeled as a first-class entity with a category type, a processing status, extracted text, a persistent file path, and a machine-readable analysis summary. Questions carry a type (multiple choice, short answer, long answer, viva, or PYQ), options with correctness flags, a reference answer and explanation, marks, difficulty, topic linkage, and—for PYQ items—the examination year. Examination sessions, per-question answers, and evaluation results complete the assessment loop, while laboratory experiments, study plans, classrooms, and enrollments cover the remaining workflows. Table I summarizes the entities.

TABLE I
CORE ENTITIES OF THE AGENTEXAM DATA MODEL

ENTITY	PURPOSE
User	Account with STUDENT or FACULTY role and target examination date
Subject	Course detected from uploads; scoped to its owner or a classroom
Unit / Topic	Curriculum hierarchy; topics carry importance, PYQ frequency, weightage
Material	Uploaded document: category, status, extracted text, analysis summary
Question	Bank item (MCQ / short / long / viva / PYQ) with marks, difficulty, topic, year
MockTest	Examination session: self-practice or faculty-assigned, with due date
Answer / Result	Per-question responses and AI evaluations with feedback
Experiment	Laboratory exercise generated from lab manuals
StudyPlan	Seven-day adaptive schedule of study tasks
Classroom / ClassroomStudent	Faculty-managed cohort and student enrollments

C. Authentication, Roles, and Visibility

Accounts authenticate with e-mail and password credentials; passwords are hashed with scrypt and sessions are issued as HS256 JSON Web Tokens valid for seven days. Roles are STUDENT and FACULTY. Subject visibility follows a single rule implemented as a shared guard: a student sees the subjects they created *or* the subjects shared into classrooms they are enrolled in, and a faculty member sees the subjects they own. Every subject-scoped endpoint—unit listing, question bank access, examination start, viva, practical, and material operations—resolves visibility before reading data and returns an authorization error otherwise. Material re-analysis and deletion are further restricted to the uploader, the subject owner, or the classroom's faculty, so that shared classroom material cannot be modified by enrolled students. Uploads are limited to 10 MB per file.

IV. UPLOAD-DRIVEN MATERIAL PROCESSING

A. Multi-Format Text Extraction

The pipeline begins with format-aware text extraction. Plain-text and Markdown-like files are read directly. PDFs are extracted with the Poppler *pdftotext* utility invoked as a child process; DOCX files are converted with a Mammoth-based extractor with a Zip-archive fallback. Image files—and PDFs that yield no text layer, which indicates scanned pages—are rendered to page images with *pdftoppm* and transcribed by a vision-language model acting as an OCR engine, an approach complementary to classical OCR pipelines [10].

The child-process choice for PDF extraction is deliberate. An earlier design used a JavaScript PDF parser imported dynamically inside the server runtime; under the application bundler, the dynamic import failed silently and every real PDF was ingested with empty extracted text—a defect that surfaced later as ungrounded questions and an empty PYQ analysis. Invoking the Poppler binary through a child process is immune to bundler transformation and restored extraction of 3,558 characters from the same scanned university paper that previously yielded zero. Extraction failures are contained: any exception returns a structured no-text result rather than aborting the pipeline, and the material is flagged for re-analysis.

B. Subject Detection and Curriculum Structuring

Extracted text is submitted to an LLM prompted to return a structured analysis: the subject name and code, its units, and for each topic an importance rating, a PYQ-frequency estimate, and an examination weightage. When the user selected an existing subject at upload time, the analysis augments that subject; otherwise the system reuses an existing subject whose name matches case-insensitively (preferring, for faculty uploads, a subject already shared into the selected classroom) or creates a new one with a collision-free course code. Structure merging is protective: units and topics are matched by name so that re-uploads and overlapping material never duplicate curriculum entries. If the LLM analysis fails or returns malformed output, a deterministic heuristic derives a placeholder structure from document headings so that the upload remains usable and can be re-analyzed later.

C. Category-Wise Organization

Every upload is classified into one of four material categories—SYLLABUS, NOTES, PYQ (previous-year papers), or LAB_MANUAL—explicitly selectable at upload time. The material library renders one group per subject with fixed-order category sections inside each group, so a student managing five or six subjects sees each subject's syllabus, notes, PYQ papers, and lab manuals as separate shelves rather than a flat chronology. Uploads whose subject is still being detected in the background appear in a transitory “detecting subject” group until the pipeline assigns one. Materials carry status badges (processing, analyzed, failed), and failed materials expose a re-analysis action; deletion is confirmation-guarded. This organization directly addresses

the multi-subject usability failure observed when a flat list mixed 30 files across six subjects.

D. Asynchronous Execution Model

Material analysis involves multiple LLM calls and typically requires 20–60 s of server work. An initial synchronous design—running the whole pipeline inside the upload request—proved fragile in three observed ways: gateway timeouts surfaced as intermittent HTTP 502 failures when a pipeline exceeded the proxy window; uploads written inside the development server's watched file tree triggered mid-request application rebuilds; and bursts of LLM calls occasionally exceeded provider rate limits. The reworked execution model (Fig. 2) decouples ingestion from analysis entirely.

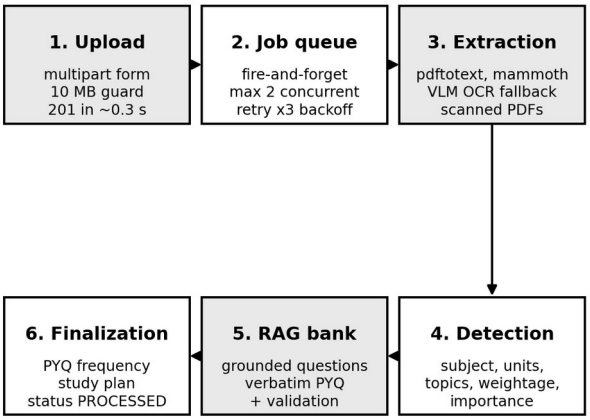


Fig. 2. Asynchronous upload-driven processing pipeline. The upload is acknowledged immediately; analysis proceeds in the background with bounded concurrency and retry with backoff.

The upload endpoint validates the file, persists it to a document store *outside* the watched application tree, creates a database row in a PROCESSING state, enqueues a background job, and returns HTTP 201 with the accepted material in approximately 0.3 s. A fire-and-forget in-process queue executes pipelines with a maximum concurrency of two, serializing LLM load while keeping latency bounded. LLM calls retry up to three times with 2 s and 5 s backoff on rate-limit or transient errors. Materials still in PROCESSING after ten minutes are presented as failed with a one-click re-analysis action, and the client's three-second status polling detects both completion and failure, translating state transitions into notifications that report the generated unit, topic, and question counts. On the client, multi-file drops are uploaded sequentially—one request completing before the next begins—which further reduces concurrent LLM pressure.

V. Retrieval-Augmented Generation

A. Corpus Construction and Chunking

Once a material reaches the analyzed state, its extracted text joins the subject corpus. The corpus is rebuilt over *every* processed material of the subject—irrespective of category—so that syllabus statements, note passages, PYQ questions, and lab-manual procedures all participate in retrieval. Text is chunked at 900 characters with a 120-character overlap, a granularity that keeps syllabus statements and individual PYQ questions intact while producing enough chunks for meaningful term statistics.

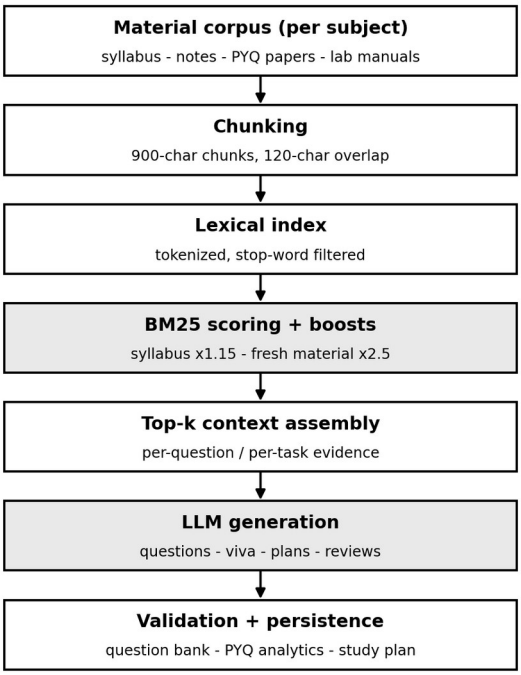


Fig. 3. Retrieval-augmented generation flow. Evidence retrieved from every processed material grounds all generative artifacts.

B. Lexical Retrieval

Retrieval scores candidate chunks with Okapi BM25 [2]. Chunks are tokenized with stop-word removal; query terms are drawn from the generation target (topic name and question-type mix). The score of a document D against query Q is

$$\text{score}(D,Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i,D)\,(k_1+1)}{f(q_i,D) + k_1\left(1 - b + b\,\frac{|D|}{\text{avgdl}}\right)} \qquad (1)$$

where f(q_i, D) is the term frequency of query term q_i, |D| the chunk length, avgdl the mean chunk length, and k1 and b the standard length-normalization parameters. Two multiplicative priors bias selection: chunks originating from SYLLABUS-category material receive a 1.15 boost, reflecting their authority over informal notes, and chunks from recently processed material receive a time-decayed boost up to 2.5, so that fresh uploads shape generation while stale content remains available. Lexical retrieval was chosen over dense embeddings [12] deliberately: course corpora are small, per-subject, and volatile (every upload changes them); BM25 is deterministic, auditable—each generated question can be traced to the chunks that produced it—and requires no embedding infrastructure or model-versioning concerns, at the cost of weaker semantic matching across paraphrase, a limitation revisited in Section IX.

C. Context-Grounded Generation

Before any generative call, a subject-context builder assembles the highest-scored chunks, together with the subject's curriculum skeleton and topic metadata, into the prompt context. Question-bank generation then requests a specified count of items spanning multiple-choice, short-

answer, long-answer, and viva types, each with options and correctness flags where applicable, a reference answer, an explanation, marks, difficulty, topic linkage, and—when evidence identifies it—an examination year. Outputs are parsed and validated: banks that fail schema validation, or that degrade to the deterministic template fallback, are retried once with a repaired prompt before the fallback is persisted and flagged. Laboratory manuals additionally trigger experiment generation when the subject has none. The same context builder serves examination assembly, viva questioning, laboratory review, and study-plan generation, making retrieval the single funnel through which the corpus influences every downstream artifact.

D. PYQ Extraction and Analysis

Previous-year papers receive a dedicated extraction prompt: the model is instructed to reproduce the paper's actual questions verbatim, preserving marks, question type, and the examination year parsed from the paper header, and to label every item with source PYQ. A structured regex fallback scans the paper text for question-shaped lines—numbered or lettered items carrying marks annotations—and persists those verbatim, so that PYQ extraction remains grounded even when the LLM call fails; a PYQ upload that yields zero PYQ-tagged questions is itself treated as a pipeline failure and retried. After each ingestion, per-topic PYQ frequencies are recomputed from the persisted evidence—the count of PYQ-tagged questions per topic—and never lowered below syllabus-derived estimates. PYQ analytics views aggregate the extracted items into year-wise and topic-wise distributions that drive an importance ranking used by the study planner and surfaced to the learner.

VI. Adaptive Assessment and Analytics

A. Examination Engine

The theory examination engine assembles a paper from the question bank under learner-selected parameters (question count, duration, and theory or mixed mode), presents multiple-choice and subjective questions with a countdown timer, an answer palette, and automatic draft saving, and supports resumption of in-progress sessions. Faculty-conducted tests reuse the same engine with a different origin: the faculty selects a subject and paper parameters, one shuffled selection of questions is drawn, and an examination row is created per enrolled student with an assigner, a title, and a due date. Students see assigned tests on their dashboard and examination view with due-date urgency indicators and start them directly; starting is idempotent so an interrupted session resumes rather than duplicating.

B. AI Evaluation of Responses

Multiple-choice items are scored objectively. Subjective answers, viva responses, and laboratory submissions are evaluated by LLM prompts that receive the question, the reference answer or rubric, and the learner's response, and return a numeric award with specific feedback and improvement suggestions. Each evaluation degrades gracefully: on provider rate limits or malformed output the system applies deterministic partial-credit rules (keyword overlap against the reference answer) so that a session always completes with a grade. In observed use, the evaluator returned zero awards with itemized feedback for placeholder answers, and a laboratory code submission was reviewed at 7/10 with concrete improvement suggestions—behavior consistent with a rubric-following grader rather than a lenient one.

C. Weak-Topic Detection

Topic performance aggregates awards across all evaluated answers of a subject. The status thresholds are calibrated to err toward surfacing deficits: a topic with a mean score of 75% or above is strong; below 60% it is weak *regardless of attempt count*—a zero after a single missed question is treated as a genuine deficit, an explicit correction of an earlier rule that marked every topic with fewer than three attempts as "insufficient data," which suppressed all weak-topic reporting for new users; scores between 60% and 74% are weak only when supported by three or more attempts, and otherwise remain insufficient. Each weak topic is reported with a recommendation grounded in the learner's actual library—for example, directing the learner to re-read the two uploaded materials that cover the topic—rather than a generic advisory.

D. Study-Plan Generation

Study planning is AI-first: an LLM composes a seven-day plan from the subject's hot topics (weightage multiplied by PYQ frequency), the learner's weak topics, and the target examination date, with a deterministic weightage-ordered template as fallback. A new plan archives the previous one, tasks are checkable and persisted day by day, and plans are regenerated automatically whenever the corpus changes materially—a new subject appears, or a fresh upload extends an existing one—so the schedule tracks the learner's actual material state.

E. Faculty Classroom Workflow

Faculty create classrooms and receive a six-character join code designed to avoid ambiguous characters. Students join by enrollment—either self-provisioned or added by the faculty, in which case an account is found or created with a temporary password that the faculty communicates. Faculty upload material directly into a classroom, where it becomes visible to every enrolled student with attribution to the uploading faculty member. Conducted tests group per-student examination rows back into a single card showing completion state, per-student scores, and the cohort average; the roster view reports each student's completed-test count, average score, and last activity. A faculty dashboard aggregates classroom count, distinct students, owned subjects, materials, conducted tests, and a recent-activity feed.

VII. System Implementation

AgentExam is implemented as a single TypeScript application on the Next.js App Router [13], with the web client and the API surface co-deployed in one process. The client is a single-page shell that switches views in place; the server exposes 27 REST endpoints implemented as route handlers with request-level authentication resolution. Persistence uses Prisma ORM [14] over SQLite with a schema of thirteen entities; uploaded files reside in a directory outside the application source tree to avoid development-server file watching. The LLM and vision-

language services are accessed through a server-side SDK and are never exposed to the client. Table II summarizes the stack.

TABLE II
TECHNOLOGY STACK OF THE DEPLOYED SYSTEM

LAYER	TECHNOLOGY
Presentation	React 19 with Next.js App Router; Tailwind CSS; shadcn/ui component system
Application	27 REST route handlers; JWT authentication (HS256); scrypt password hashing
Intelligence	Hosted LLM and vision-language service via server-side SDK; BM25 retrieval engine
Extraction	Poppler pdftotext and pdftoppm; Mammoth (DOCX); VLM OCR for scanned pages
Data	Prisma ORM over SQLite (13 entities); document store outside the build tree
Concurrency	In-process job queue, maximum 2 concurrent pipelines, retry with backoff

The deployment target is a single server process; the job queue is intentionally in-process, sized for individual learners and single-classroom cohorts rather than institutional scale. All AI integrations follow a common pattern—typed prompt construction, structured-output validation, bounded retries, and deterministic fallbacks—which concentrates failure handling in one place and keeps the system operational through provider outages.

VIII. EVALUATION

A. Methodology

The evaluation reports measured behavior of the deployed system rather than an offline benchmark. The live instance was exercised end-to-end through browser automation covering the full user journeys (registration, upload, examination, viva, practical, study plan, classroom administration) and through a REST harness for endpoint-level checks. A corpus of authentic material was used: a student user's real uploads, consisting of two Cloud Computing (course code CSE 021) papers and one Design and Analysis of Algorithms (CSE 356) paper, alongside seeded demonstration data for classroom workflows. All timings are wall-clock measurements from the running server.

B. Pipeline Behavior

The asynchronous design changes the latency profile materially. The upload endpoint acknowledges in approximately 0.3 s—compared with 40–60 s for the earlier synchronous design, during which gateway timeouts produced intermittent 502 failures—and background analysis completes in 20–25 s per document. Text extraction from the scanned CSE 021 PDF yields 3,558 characters where the bundled-parser defect had yielded zero. A complete first-session onboarding—uploading a machine-learning syllabus as a new student—produced a detected subject, five units, twenty topics, a fourteen-question bank, and a seven-day study plan without any manual configuration, and the dashboard transitioned from its onboarding state to the full analytics layout upon completion.

C. Generation Quality

Fig. 4 shows the composition of the generated question banks on the live system for the authentic corpus: Cloud Computing accumulated 56 questions of which 17 were PYQ-tagged, and Design and Analysis of Algorithms 32 questions of which 18 were PYQ-tagged; 35 distinct verbatim PYQ items in total were surfaced from the user's papers—for example, “Relate cloud computing and mobile computing” and “What are different asymptotic notations?”—each carrying year and marks metadata. PYQ analytics views rendered year-wise and topic-wise distributions from these items. Weak-topic detection, previously empty for this user, surfaced eight deficit topics with material-grounded recommendations, and the regenerated study plan referenced the learner's actual weak topics and uploaded materials.

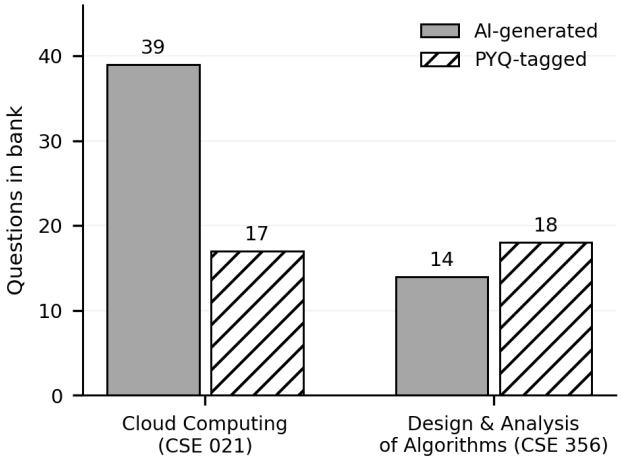


Fig. 4. Composition of the generated question banks on the live system (measured on the authentic corpus). PYQ-tagged items were extracted verbatim from the user's previous-year papers.

D. Defect Resolution

Because the reported defects were reproduced, root-caused, and re-measured on the live system, they constitute an engineering evaluation in themselves. Table III maps each user-reported defect to its root cause, the resolution, and the measured outcome.

TABLE III
USER-REPORTED DEFECTS, ROOT CAUSES, RESOLUTIONS, AND MEASURED OUTCOMES

REPORTED DEFECT	ROOT CAUSE AND RESOLUTION	MEASURED OUTCOME
Weak-topic detection returned empty results	Status logic marked all topics with <3 attempts as insufficient; thresholds revised so scores below 60% are weak regardless of attempts	8 weak topics surfaced with material-grounded recommendations
Intermittent upload failures (HTTP 502)	40–60 s synchronous pipeline, watcher-triggered rebuilds, provider rate limits; replaced by async queue, storage outside watched tree, retries with backoff	0.3 s acknowledgment; 20–25 s background completion
Materials across 5–6 subjects mixed together	Flat, ungrouped material listing; replaced with subject groups and fixed-order category sections	All subjects organized per category (syllabus / notes / PYQ / lab)

Reported defect	Root cause and resolution	Measured outcome
PYQ analysis page empty	Bundled PDF parser yielded no text and no PYQ-tagged questions; child-process extraction, verbatim extraction prompt, regex fallback	35 verbatim PYQs extracted from authentic papers
Generated content not grounded in uploads	No retrieval context fed to generation; RAG layer introduced over every material with source and recency boosts	17–18 PYQ-tagged questions per subject; plans cite real materials

E. Classroom Workflow

The faculty workflow was exercised with a three-student classroom sharing two subjects and five materials. A conducted unit test on Cloud Computing was completed by two of three students with scores of 81% and 54% (cohort average 67.5% on the test), aggregated correctly on the faculty dashboard alongside classroom, material, and activity statistics. Student-side, enrolled students saw classroom-sourced subjects and faculty-attributed material, and assigned tests appeared on their dashboards with due dates. Add-student provisioning, join-code creation, and per-student removal were verified through the interface.

IX. Discussion and Limitations

Retrieval. BM25 grounds generation in exact term overlap; paraphrased evidence and synonym-heavy notes are under-retrieved. A hybrid of lexical and dense retrieval [12], with per-subject embedding caches invalidated on upload, is the natural extension; the auditability argument for pure lexical scoring weakens as corpora grow.

Evaluation validity. LLM-assigned marks for subjective answers are convenient but not calibrated against human graders; the rubric prompt bounds but does not eliminate variance. The faculty review affordances—per-question feedback inspection and result visibility—are the current human-in-the-loop safeguard, and a graded-agreement study against multiple human markers remains future work.

Scale. The in-process queue and SQLite storage are sized for individual learners and single classrooms. Institutional deployment would require a durable job queue, a relational server, and per-tenant storage isolation; nothing in the architecture precludes this, but it is untested.

Integrity and privacy. The platform augments practice, not proctoring: unsupervised mock examinations certify nothing. Uploaded material may contain copyrighted course content; the deployment stores it server-side for the uploading account's use, and a production deployment should add retention policy and encryption at rest. Finally, generative content can still be subtly wrong even when grounded; the validation layer rejects malformed structure but not subtle factual error, which motivates keeping faculty review in the loop.

X. Conclusion and Future Work

AgentExam demonstrates that a full adaptive-assessment loop can be bootstrapped from nothing but the learner's own material. The system reconstructs curriculum structure from uploads, organizes artifacts category-wise per subject, grounds every generative artifact in corpus retrieval, and closes the loop with AI-evaluated practice, weak-topic detection, and study planning—while faculty operate classrooms with shared libraries, assigned tests, and per-student analytics over the same machinery. Measured behavior on authentic university material—sub-second acknowledgment, 20–25 s analysis, 35 verbatim PYQ extractions, and eight surfaced deficit topics—indicates that the approach is practical with current hosted LLMs.

Future work proceeds on four axes: hybrid dense–lexical retrieval with per-subject embedding caches; calibrated evaluation, comparing AI-assigned marks against multiple human graders and surfacing disagreement; spaced-repetition scheduling in the spirit of trainable practice models [7] to time reviews beyond the current seven-day horizon; and multilingual OCR and generation, extending the vision-model extraction path to non-English papers. A controlled study with a student cohort, measuring practice outcomes against a baseline platform, remains the decisive validation of the learning claims.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pp. 9459–9474.

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.

[4] OpenAI, "GPT-4 technical report," *arXiv preprint* arXiv:2303.08774, 2023.

[5] K. VanLehn, "The behavior of tutoring systems," *International Journal of Artificial Intelligence in Education*, vol. 16, no. 3, pp. 227–265, 2006.

[6] P. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *International Journal of Artificial Intelligence in Education*, vol. 30, no. 1, pp. 121–204, 2020.

[7] B. Settles and B. Meeder, "A trainable spaced repetition model for language learning," in *Proc. 54th Annu. Meeting of the Association for Computational Linguistics (ACL)*, Berlin, Germany, 2016, pp. 1848–1858.

[8] P. Black and D. Wiliam, "Assessment and classroom learning," *Assessment in Education: Principles, Policy & Practice*, vol. 5, no. 1, pp. 7–74, 1998.

[9] E. Kasneci, K. Seidel, S. Kämmel, M. Groß, D. Leisenberg, D. Böhler, C. Häußer, A. Bär, E. Fischer, P. Häring, and C. Gress, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, art. 102274, 2023.

[10] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. Ninth Int. Conf. on Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.

[11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022, pp. 24824–24837.

[12] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. on Empirical*

Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[13] Vercel Inc., “Next.js: The React framework,” 2026. [Online]. Available: https://nextjs.org

[14] Prisma, “Prisma ORM: type-safe database access,” 2026. [Online]. Available: https://www.prisma.io